

Тема: «Разработка и реализация SaaS-платформы для автоматизированного создания легких сайтов-визиток на базе стека JavaScript»

Название проекта: BobbyHost Generator

Введение

SaaS (Software as a Service) — это модель распространения ПО, при которой приложение размещено в облаке поставщиком и доступно клиентам через интернет по подписке. За клиентами остается настройка их данных и пользователей, а поставщик отвечает за хостинг, обновления, безопасность и масштабирование. У меня прототип генератора CMS в виде SaaS-услуги, потому что нет платной подписки. Ниже описал разницу.

1) Генератор как локальный инструмент (CLI/npm-пакет).

Это не SaaS. Мечта - создать инструмент, который пользователи устанавливать и запускать у себя локально (или в CI). Он генерирует код/конфигурацию CMS, но сам процесс генерации не проживает в облаке и не управляется как сервис.

Преимущества: simplicity, контроль пользователя, нет нюансов мультиарендности.

Недостатки: пользователь сам отвечает за развёртывание сгенерированной CMS, обновления и безопасность.

2) Генератор как SaaS-услуга.

Это полноценный SaaS: онлайн-инструмент, который позволяет пользователю сконфигурировать нужную CMS-шаблон и автоматически разворачивать готовый экземпляр CMS под клиента в облаке, например на vps, который находится в облачном кластере.

Клиент платит подписку за доступ к сервису, а поставщик отвечает за хостинг, обновления, безопасность, бэкапы, мониторинг и масштабирование.

Преимущества: быстрая постанова в развёрнутом виде, единая политика безопасности и обновлений, возможность централизованной поддержки и монетизации.

Недостатки: сложнее реализовать из-за мультиарендности, требований к данным клиентов, SLA, миграциям и соответствию требованиям по безопасности.

1. Актуальность темы:

Используя такие cms, как WordPress, Joomla для себя и как практикант службы техподдержки одного хостинга, изучив основы JavaScript на курсах выдающегося преподавателя господина Сизова, я пришел к выводу:

1) распространенные CMS (системы управления контентом для создания сайтов) на языке PHP - WordPress, Joomla, Drupal требовательны к серверным ресурсам

из-за своей монолитности, особенно из-за плагинов и расширений. Нужно специально настраивать различные связки Apache+Nginx+f-m-php+Varnish+Redis. А еще эти системы имеют критические уязвимости и требуется защищать их либо файерволами L7-уровня типа WAF типа ModSecurity2 для Apache Или Nginx или прописывать в конфиги Apache и Nginx Защиту против взлома таких cms.

2) С другой стороны, существующие современные CMS на стеке JavaScript (такие как Strapi, Ghost, KeystoneJS или Netlify CMS) обеспечивают высокую скорость работы и более высокий уровень безопасности. Однако, я их юзал и они реально сложнее в управлении, чем тот же WordPress, потому что нужно дописывать код под нужный функционал и там нет плагинов в большинстве случаев типа.

Проект BobbyHost Generator нужен как раз для того, чтобы объединить высокую производительность и безопасность JavaScript-архитектуры с предельной простотой интерфейса и уйти от php-реализации cms. Это специализированный генератор, который исключает избыточность классических CMS, предлагая пользователю защищенный и легкий продукт без необходимости администрирования сложных систем.

2. Целевая аудитория:

Проект ориентирован на пользователей, которые не отягощают себя знаниями в области php, javascript и не хотят платить много за виртуальный хостинг, который подорожал для php-сайтов.

Для кого мой проект: для тех, кто хочет за 50 руб разместить сайт на виртуальном хостинге чтобы в тарифном плане хостинга размер дискового пространства был не 1-2 Гб, а максимум 300 Мб. Соответственно и потребляемые ресурсы хостинга этим сайтом были намного меньше, чем у сайта, написанного на php.

3. Предполагаемые функциональные возможности:

Система авторизации: защищенный вход в личный кабинет на базе Node.js.

Конструктор сайта: минималистичный интерфейс для ввода персональных данных, исключающий техническую сложность управления контентом.

Автодеплой (Автоматизированное развертывание): генерация статической или динамической страницы-визитки на основе введенных данных.

Управление доменами: возможность выбора уникального поддомена в зоне bobbyhost.ru с автоматической регистрацией пути в базе данных.

Безопасность: отсутствие административной панели на стороне фронтенда, что минимизирует атаки уровня L7 и SQL-инъекций.

4. Используемые технологии:

Выбор стека обусловлен стремлением к максимальной легкости и безопасности решения:

Runtime: Node.js — обеспечивает высокую скорость обработки запросов при минимальном потреблении оперативной памяти по сравнению с Apache+PHP.

Backend фреймворк: Express.js — для реализации быстрой маршрутизации и связи с БД.

Frontend: Vanilla JavaScript — отказ от тяжелых библиотек для мгновенной загрузки страниц на мобильных устройствах.

База данных: SQLite — для эффективного хранения данных пользователей и доменных имен без накладных расходов, свойственных более крупным СУБД.

5. Ожидаемый результат:

Итогом работы станет SaaS-сервис на домене bobbyhost.ru, представляющий собой альтернативу CMS на php и сложным CMS на javascript.

Ожидаемые показатели:

1. **Производительность:** скорость загрузки сгенерированных страниц в 3-4 раза выше, чем у аналогичных визиток на WordPress.
2. **Безопасность:** стойкость к эксплойтам, с помощью которых эксплуатируются уязвимости на PHP-сайтах типа WordPress.
3. **Удобство:** время от регистрации до получения готового сайта по адресу имя.bobbyhost.ru составит менее 2 минут при нулевой технической подготовке пользователя.

Технологическое обоснование в Приложении №1.

Технологическое обоснование и протокол развертывания SaaS-платформы «BobbyHost Generator»

1. Обоснование технологического стека (программного слоя, как выражается один мой знакомый главный сисадмин):

Выбор стека технологий продиктован необходимостью создания **LWS (Lightweight Website)** — архитектуры, которая была бы лишена недостатков тяжелых WordPress и Joomla и сложности JS-систем на продакшен.

1.1. Серверная часть (Backend): Node.js и Express

Производительность: в отличие от PHP, который создает новый процесс для каждого HTTP-запроса (модель блокирующего ввода-вывода), Node.js работает на движке V8 и использует событийно-ориентированную неблокирующую модель. Это позволяет обрабатывать тысячи одновременных подключений при минимальном потреблении оперативной памяти.

Безопасность: использование JavaScript на сервере позволяет избежать специфических для PHP уязвимостей (таких как RFI/LFI или небезопасная десериализация), а отсутствие готовых «админок» с известными путями доступа снижает риск атак методом перебора (Brute-force).

Единый язык: использование JS на обоих концах (Fullstack JS) ускоряет разработку и упрощает структуру проекта.

1.2. База данных: SQLite

Легкость: SQLite — это встраиваемая СУБД, не требующая отдельного серверного процесса. Вся база данных хранится в одном файле.

Скорость: для задач генерации простых визиток скорость чтения из файла SQLite превышает скорость обращения к сетевому сокету MySQL/PostgreSQL.

Портабельность: файл database.db легко копировать и восстанавливать, что критично для MVP.

1.3. Клиентская часть (Frontend): Vanilla JS

Zero-Dependency: отказ от React/Vue/Angular обоснован тем, что визитка должна загружаться мгновенно (менее 1 секунды). Чистый JavaScript обеспечивает минимальный размер бандла.

SEO-эффективность: генерируемый HTML-код максимально прост, что идеально для индексации поисковыми роботами.

2. Этапы развертывания на хостинге (задеплоить на хост)

Развертывание планируется на VPS (Virtual Private Server) под управлением ОС AlmaLinux 9.7 на домене bobbyhost.ru.

1: Подготовка окружения сервера

1. **Обновление системы:** `sudo yum upgrade`.
2. **Установка Node.js:** версия LTS (например, v20.x).
3. **Установка менеджера процессов PM2:** `npm install pm2 -g`. PM2 – этой фичей обеспечиваем перезапуск приложения в случае сбоя или перезагрузки сервера.

2. Клонирование и настройка приложения

1. **Загрузка кода:** Клонирование репозитория через Git: `git clone https://github.com/user/bobbyhost.git` (пока не создал эту репку).
2. **Установка зависимостей:** переход в папку проекта и выполнение `npm install`.
3. **Настройка окружения:** создание файла `.env` для хранения секретных ключей авторизации.

3. Запилить конфиг web-сервера (Nginx)

Для того чтобы приложение было доступно по протоколу HTTP/HTTPS на стандартных портах, используется Nginx в качестве **Reverse Proxy**.

1. Входим на дедик (выделенные сервер) сервер или vps.
2. Устанавливаем web-сервер Nginx: `sudo yum install nginx`.

Создание конфига web-сервера

```
server {
    listen 80;
    server_name bobbyhost.ru *.bobbyhost.ru;

    location / {
        proxy_pass http://ip:3000; # Перенаправление на Node.js тут мой ip vps или
        localhost домашнего компа.
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
```

```
}  
}
```

.bobbyhost.ru — это позволяет динамически обрабатывать поддомены пользователей.

Настройка HTTPS (SSL) Получение бесплатного SSL-сертификата
Безопасность авторизации пользователей обеспечивается сертификатом Let's Encrypt.

1. Установите EPEL:

```
sudo dnf install -y epel-release
```

Установите Certbot и плагин nginx:

```
sudo dnf install -y certbot python3-certbot-nginx
```

или команда в одну строку:

```
sudo dnf install -y epel-release && sudo dnf install -y certbot python3-certbot-nginx
```

Добавить обновление:

```
sudo dnf update -y
```

2. Получение сертификата: `sudo certbot certonly --manual --preferred-challenges dns -d bobbyhost.ru -d *.bobbyhost.ru.`

4. Запуск системы

1. **Инициализация БД:** Запуск скрипта миграции для создания таблиц SQLite.

2. **Старт приложения через PM2:**

```
pm2 start server.js --name "bobbyhost-gen"  
pm2 save  
pm2 startup
```

5. Настройка Wildcard DNS

Чтобы система могла мгновенно выдавать домены вида name.bobbyhost.ru, в панели управления DNS домена добавляется **A-запись:**

Хост: *

Значение: IP-адрес моего vps или выделенного сервера сервера.

Резюме

Выбранный стек (Node.js + SQLite + Nginx) позволяет создать отказоустойчивую и дешевую в обслуживании систему.

В отличие от PHP-хостинга, где каждый сайт требует ресурсов, данная архитектура позволяет содержать тысячи визиток в рамках одной базы данных и одного серверного процесса, потребляя при этом не более 200-300 МБ ОЗУ в режиме ожидания.

Это моя жалкая ученическая попытка, которую нельзя расценивать, как претензию на какую-то крутую работу. Мне почти не дается JavaScript, нужно было пойти на сисадмина, мне проще это. Просто я пытаюсь не позорить своего великого и гениального преподавателя господина Сизова. Я – всего лишь, высокофункциональный аутист. И я восхищаюсь, с какой легкостью господин Сизов знает и преподает JavaScript – там есть такие лекции, как будто системный архитектор рассказывает, что он написал в работах по философии. Мне кажется, что он с такой же легкостью мог бы освоить квантовую теорию поля или диаграммы Феймана.